Übung 9: Sortieralgorithmen

(Dauer: 90 Minuten - Gruppenarbeit empfohlen)

Ziel der Übung: Sie können verschiedene Sortieralgorithmen anwenden und hinsichtlich ihrer Laufzeit, ihres Speicherbedarfs und ihrer Stabilität vergleichen. Sie können für unterschiedliche Anwendungsszenarien geeignete Sortierverfahren auswählen und ihre Wahl begründen. Darüber hinaus analysieren Sie die Laufzeit von Algorithmen, erkennen grundlegende Entwurfsmuster und nutzen geeignete Datenstrukturen, um Algorithmen effizienter zu gestalten.

1. Führen Sie die folgenden Sortieralgorithmen auf den angegebenen Eingabefolgen aus. Sortieren Sie jeweils in aufsteigender Reihenfolge. Dokumentieren Sie die wesentlichen Zwischenschritte des jeweiligen Algorithmus.
   (a) Insertionsort für 0, 4, 2, 7, 6, 1, 3, 5.
   (b) Selectionsort für 0, 4, 2, 7, 6, 1, 3, 5.
   (c) In-place Heapsort für 0, 6, 2, 7, 4. (Sie können den Heap auch aufzeichnen. Stellen Sie sicher, dass der erste Schritt, den Sie machen der Schritt buildHeap ist!)
   (d) Mergesort für 0, 4, 2, 7, 6, 1, 3, 5.
   (e) Quicksort für 18, 7, 22, 34, 99, 19, 11, 4. Wählen Sie bei jedem Partitionierungsschritt das erste Element des betrachteten Teilarrays als Pivotelement und dokumentieren Sie die einzelnen Partitionierungsschritte.

2. Geben Sie für die folgenden Szenarien alle Sortieralgorithmen an, die am besten funktionieren würden. Begründen Sie Ihre Antwort kurz. Zur Wahl stehen (es kann auch passieren, dass keiner der Algorithmen funktioniert):

   Insertionsort, In-place Heapsort, Mergesort, Selectionsort, Quicksort.

   (a) Für das Sortieren von n Elementen wird eine Laufzeit von $\Theta(\log n)$ im Best Case gefordert. Welche der genannten Sortieralgorithmen erfüllen diese Anforderung?
   (b) Wir möchten Doubles sortieren mit einer Laufzeit von $\Omega(n \log(n))$ für einen Best Case Fall.
   (c) Sie entwickeln einen Algorithmus für einen Mars-Rover, der jede Nacht die gesammelten Daten sortieren soll. Der Rechner des Rovers verfügt nur über sehr wenig zusätzlichen Speicher und die Laufzeit des Sortieralgorithmus soll möglichst gut sein. Welchen der genannten Sortieralgorithmen würden Sie bevorzugen? Begründen Sie Ihre Wahl unter Berücksichtigung von Laufzeit und zusätzlichem Speicherbedarf.
   (d) Bei Klassenfotos werden die Kinder nach ihrer Körpergröße sortiert. Dabei ist die ursprüngliche Reihenfolge der Kinder nicht relevant. Der Fotograf möchte die Kinder möglichst schnell sortieren. Für den Sortieralgorithmus steht ausreichend zusätzlicher Platz im Raum zur Verfügung. Welchen der genannten Algorithmen würden Sie bevorzugen? Begründen Sie Ihre Wahl.
   (e) Angenommen, wir haben ein Array mit ein paar hundert Elementen, das fast vollständig sortiert ist, abgesehen von ein oder zwei Elementen, die mit dem vorherigen Element in dem Array vertauscht wurden. Welcher Algorithmus eignet sich am besten, um diese Daten zu sortieren?

3. Sie wollen im Sommer eine Tour zwischen den Geburtsstätten der wichtigsten deutschen Musiker machen. Sie möchten die Tour mit dem Kanu durchführen. Eine Karte von Deutschland mit allen Gewässern und Flüssen sei Ihnen gegeben.

   Um sich auf die Musiker einzustimmen, möchten Sie während der Kanutour deren Lieder in chronologischer Reihenfolge hören von alt nach neu. Auf dem Weg zu einem Ziel möchten Sie nur Lieder des einen Künstlers hören, der am Zielort geboren ist. Während der Tour möchten Sie weitere Lieder der Musiker herunterladen, deren Geburtsort Sie noch nicht besucht haben. Nehmen Sie an, dass Sie die Musiksammlung als Tabelle abspeichern, die Sie als Array implementieren. Als Schlüssel der Schlüssel-Wert-Paare wählen Sie Nach- und Vorname des Künstlers inkl. Songtitel. Werte sind bspw. Künstlername, Songtitel, Erscheinungsdatum, Pfad zur MP3-Datei, ...

   (a) Welchen Sortieralgorithmus könnten Sie idealerweise nutzen, um die Lieder wie gewünscht zu sortieren? Welche Eigenschaften muss der Algorithmus haben? Bedenken Sie, dass Sie auf dem Kanu nur einen sehr kleinen Rechner dabei haben. Begründen Sie Ihre Antwort. Diese Frage hat mehrere korrekte Antworten.

   (b) Welchen Sortieralgorithmus könnten Sie nutzen, um gerade neu heruntergeladene Lieder möglichst effizient in die bereits sortierte Liedersammlung einzufügen? Bitte kurz begründen.

   (c) Wählen Sie eine Datenstruktur mit der Sie die Reiseroute planen können. Wie werden Geburtsstädte, Wasserwege, Entfernungen zwischen den Geburtsstädten in der Datenstruktur gespeichert/modelliert?

   (d) Mit welchem Algorithmus können Sie den kürzesten Pfad zwischen zwei Geburtsstädten, gegeben Ihre Datenstruktur aus (c), berechnen?

   (e) Mit welchem Algorithmus können Sie überprüfen, ob alle Geburtsstädte über die vorhandenen Wasserwege voneinander erreichbar sind? Die Karte der Wasserwege sei Ihnen als Graph gegeben. Begründen Sie Ihre Wahl des Algorithmus.

4. Sie sehen im folgenden eine ineffiziente Implementierung eines Algorithmus zur Suche von Duplikaten in einer Liste und eine effiziente Implementierung.

   Ineffizient:

```
def has_duplicates_naive(arr):
  n = len(arr)
  for i in range(n):
    for j in range(i + 1, n):
      if arr[i] == arr[j]:
        return True
  return False

# Beispielaufruf
my_list = [1, 2, 3, 4, 5, 2]
result = has_duplicates_naive(my_list)
print(f"Gibt es Duplikate? {result}")
```

   Effizient:

```
def has_duplicates_optimized(arr):
  seen = set()
  for element in arr:
    if element in seen:
      return True
    seen.add(element)
  return False

# Beispielaufruf
my_list = [1, 2, 3, 4, 5, 2]
result = has_duplicates_optimized(my_list)
print(f"Gibt es Duplikate? {result}")
```

   (a) Welche der drei in der Vorlesung besprochenen Entwurfsmuster wurde hier genutzt, um den Algorithmus effizienter zu machen? Entwurfsmuster: Iterative Verbesserung einer Invarianten, Verwendung einer Datenstruktur, Divide and Conquer.

   (b) Mit welcher Datenstruktur könnte man den ADT Set (Menge) möglichst effizient implementieren? Geben Sie die erwartete Laufzeit für das Suchen und Einfügen eines Elements an.

   (c) Analysieren Sie die Best-Case- und Worst-Case-Laufzeit beider Algorithmen. Geben Sie für jede Situation eine möglichst genaue asymptotische Laufzeit an. Geben Sie außerdem den zusätzlichen Speicherbedarf beider Algorithmen an.

5. In einer Schule sollen die Schülerinnen und Schüler nach ihren Noten in aufsteigender Reihenfolge sortiert werden. Die Schülerinnen und Schüler sind vor der Sortierung bereits nach Klassenstufe aufsteigend angeordnet. Die Noten sind ganze Zahlen im Bereich von 0 bis 100.

   Bei identischen Noten soll die Reihenfolge nach Klassenstufe erhalten bleiben, d.,h. Schülerinnen und Schüler aus einer niedrigeren Klassenstufe sollen vor Schülerinnen und Schülern aus einer höheren Klassenstufe stehen.

   Diskutieren Sie, welche der folgenden Sortieralgorithmen für diese Anwendung geeignet sind und welche nicht. Begründen Sie Ihre Entscheidung insbesondere anhand der Stabilität der Algorithmen.

   1. Mergesort
   2. Quicksort
   3. Heapsort
   4. Insertionsort

5. Selectionsort

6. Betrachten Sie die in der Vorlesung behandelte Implementierung von Heapsort.
   (a) Bestimmen Sie die Best-Case-Laufzeit von Heapsort. Begründen Sie Ihre Antwort anhand der einzelnen Phasen des Algorithmus. Analysieren Sie insbesondere die Laufzeit von `buildHeap` und der anschließenden Sortierphase.
   (b) Geben Sie ein Beispiel für eine Eingabe an, bei der der Best Case erreicht wird.

7. Spezielles Bucketsort für Doubles: Sie möchten n Double Werte mit Bucketsort sortieren und erstellen dafür K Behälter. Jeden Behälter sortieren Sie mit dem folgenden Sortieralgorithmus:

```
from itertools import permutations

def is_sorted(arr):
  return all(arr[i] <= arr[i + 1] for i in range(len(arr) - 1))

def specialsort(arr):
  for permutation in permutations(arr):
    if is_sorted(permutation):
      return list(permutation)

# Beispielaufruf
input_list = [3, 1, 4, 1, 5, 9, 2, 5]
sorted_list = specialsort(input_list)
```

   Hinweis: Die `permutations` Funktion gibt alle möglichen Anordnungen der Elemente im Array `arr` als Tupel zurück. Für das Array `[5, 3, 6]` sind es die folgenden $n!$ Anordnungen:

```
(3, 5, 6)
(3, 6, 5)
(5, 3, 6)
(5, 6, 3)
(6, 3, 5)
(6, 5, 3)
```

   Hinweis 2: Die Funktion `permutations(arr)` erzeugt die Permutationen *lazy*, d. h. sie werden nacheinander erzeugt und nicht alle gleichzeitig im Speicher abgelegt. Die Erzeugung jeder einzelnen Permutation benötigt dabei $\Theta(n)$ Zeit.

   (a) Markieren Sie das Entwurfsmuster das in dem Sortieralgorithmus `specialsort` genutzt wird.
      A. Iterative Verbesserung einer Invarianten
      B. Benutzung einer Datenstruktur
      C. Teile und Beherrsche
      D. Brute-Force
      E. Greedy Algorithmus
      F. Dynamische Programmierung

   (b) Bestimmen Sie die Best-Case- und Worst-Case-Laufzeit von `specialsort`. Berücksichtigen Sie dabei sowohl die Anzahl der erzeugten Permutationen als auch die Laufzeit von `is_sorted`.

   Bestimmen Sie anschließend die Best-Case- und Worst-Case-Laufzeit des beschriebenen Bucketsort-Verfahrens, bei dem jeder Bucket mit `specialsort` sortiert wird. Geben Sie Ihre Ergebnisse in möglichst genauer asymptotischer Form an.

8. Wählen Sie für jedes der folgenden Szenarien den passenden Sortieralgorithmus und begründen Sie Ihre Wahl in einem kurzen Satz.

   Zur Wahl stehen: Insertionsort, Selectionsort, Mergesort, Quicksort, Heapsort, Bucketsort

   (a) Sie schreiben ein Programm, das die Bewerber für ein prestigeträchtiges Stipendium sortiert. Jeder Bewerbung wird eine numerische Punktzahl zugewiesen, um eine geordnete Rangfolge der Bewerbungen zu erstellen. Wenn zwei Bewerbungen die gleiche Punktzahl haben, wird die Bewerbung ausgewählt, die zuerst eingegangen ist.

   (b) Sie sind eine Triage-Schwester und für die Warteschlange in der Notaufnahme einer Klinik zuständig, die bestimmt, wer zuerst zum Arzt geht. Sie führen eine einzige sortierte Liste in einer Tabellenkalkulation, und wenn neue Patienten eintreffen, fügen Sie sie entsprechend dem Schweregrad ihrer Verletzung in die Warteschlange ein. Wenn zwei Patienten den gleichen Schweregrad haben, sollte der Patient, der zuerst eintrifft, den Arzt zuerst sehen.

   (c) Ein Softwareunternehmen möchte die Performance seiner Mitarbeiter für ein bestimmtes Projekt evaluieren. Die Performance-Werte der Mitarbeiter liegen zwischen 0 und 10 (Ganzzahlen), wobei es sehr viele Mitarbeiter gibt. Sie möchten die Performance-Werte effizient sortieren, um die besten und schlechtesten Leistungen leicht identifizieren zu können.

Dieses Übungsblatt ist Teil der Lehrveranstaltung *„Algorithmik“*.